Hill - Climbing Search Algorithm

```
def cost(state):
    """Calculates the number of attacking queen pairs (diagonal
collisions)."""
    c = 0
    n = len(state)
    for i in range(n):
        for j in range(i + 1, n):
            if abs(state[i] - state[j]) == abs(i - j):
                c += 1
    return c

def display_board(state):
    """Prints an n x n chessboard visualization of the state."""
    n = len(state)
    for i in range(n):
        row = ""
        for j in range(n):
            if state[j] == i:
                row += "Q "
            else:
                row += ". "
        print("  " + row)

def hill_climbing_traced(initial_state):
    state = list(initial_state)
    n = len(state)
    step = 0

    print("\n--- STARTING HILL-CLIMBING SEARCH ---")

    while True:
        current_cost = cost(state)
        print(f"\n[Step {step}] Current State: {state} | Cost: {current_cost}")
```

```
        display_board(state)

        # Check if we reached a goal state
        if current_cost == 0:
            print(f"\n>>> Goal state reached in {step} transitions!")
            return state

        neighbors = []
        # Generate swap neighbors (loop order guarantees tie-breaking by
lowest column indices)
        for i in range(n):
            for j in range(i + 1, n):
                new_state = state.copy()
                new_state[i], new_state[j] = new_state[j], new_state[i]
                neighbors.append((new_state, (i, j)))

        # Evaluate neighbors
        print("\nEvaluating neighbors:")
        best_state = None
        best_cost = float('inf')
        best_swap = None

        for idx, (neighbor, swap_pair) in enumerate(neighbors):
            c = cost(neighbor)
            print(f"  Neighbor {idx+1}: Swap cols {swap_pair} -> State:
{neighbor} | Cost: {c}")

            # Find minimum cost neighbor (first match breaks ties naturally)
            if c < best_cost:
                best_cost = c
                best_state = neighbor
                best_swap = swap_pair

        # Termination condition: stop if no strictly better neighbor exists
        if best_cost >= current_cost:
```

```
            print(f"\n>>> Algorithm terminated (Stuck at local minimum/plateau
with cost {current_cost}).")
            return state

        # State transition
        print(f"\n--> Moving to best neighbor: {best_state} (Swapped cols
{best_swap}, Cost decreased from {current_cost} to {best_cost})")
        state = best_state
        step += 1

# --- Execution with User Input ---
if __name__ == "__main__":
    user_input = input("Enter initial state array as comma-separated row
numbers (e.g., 3,1,2,0): ")
    initial_state = [int(x.strip()) for x in user_input.split(",")]

    solution = hill_climbing_traced(initial_state)
    print("\nFinal Result State:", solution)
    print("Final Cost:", cost(solution))
```

Output

```
Enter initial state array as comma-separated row numbers (e.g., 3,1,2,0): 3,1,2,0

--- STARTING HILL-CLIMBING SEARCH ---

[Step 0] Current State: [3, 1, 2, 0] | Cost: 2
  . . . Q
  . Q . .
  . . Q .
  Q . . .

Evaluating neighbors:
  Neighbor 1: Swap cols (0, 1) -> State: [1, 3, 2, 0] | Cost: 1
  Neighbor 2: Swap cols (0, 2) -> State: [2, 1, 3, 0] | Cost: 1
  Neighbor 3: Swap cols (0, 3) -> State: [0, 1, 2, 3] | Cost: 6
  Neighbor 4: Swap cols (1, 2) -> State: [3, 2, 1, 0] | Cost: 6
  Neighbor 5: Swap cols (1, 3) -> State: [3, 0, 2, 1] | Cost: 1
  Neighbor 6: Swap cols (2, 3) -> State: [3, 1, 0, 2] | Cost: 1

--> Moving to best neighbor: [1, 3, 2, 0] (Swapped cols (0, 1), Cost decreased from 2 to 1)

[Step 1] Current State: [1, 3, 2, 0] | Cost: 1
  . . . Q
  Q . . .
  . . Q .
  . Q . .

Evaluating neighbors:
  Neighbor 1: Swap cols (0, 1) -> State: [3, 1, 2, 0] | Cost: 2
  Neighbor 2: Swap cols (0, 2) -> State: [2, 3, 1, 0] | Cost: 2
  Neighbor 3: Swap cols (0, 3) -> State: [0, 3, 2, 1] | Cost: 4
  Neighbor 4: Swap cols (1, 2) -> State: [1, 2, 3, 0] | Cost: 4
  Neighbor 5: Swap cols (1, 3) -> State: [1, 0, 2, 3] | Cost: 2
  Neighbor 6: Swap cols (2, 3) -> State: [1, 3, 0, 2] | Cost: 0

--> Moving to best neighbor: [1, 3, 0, 2] (Swapped cols (2, 3), Cost decreased from 1 to 0)
```

```
[Step 2] Current State: [1, 3, 0, 2] | Cost: 0
  . . Q .
  Q . . .
  . . . Q
  . Q . .

>>> Goal state reached in 2 transitions!

Final Result State: [1, 3, 0, 2]
Final Cost: 0
```